CURRENT PROJECT AUTHORITY

Vanni Ride Master Specification

Current As Built System and Project Authority

Version 2.0

Updated 14 October 2026

Baseline commit 6c1719b

University of Vavuniya | IT3162 Group 6

This document defines the implemented Vanni Ride system, the rules that all project components must follow, the current free deployment, and the work still required before public release. It supersedes unresolved pre-development assumptions wherever they conflict with the working application.

Do not include passwords, API keys, JWT secrets, or database connection strings in this document.

Document Control

This specification is the governing reference for continued development, team review, testing, and deployment. The repository implementation at the stated baseline is the evidence for as-built behavior. Future approved changes must update both the code and this document.

Field	Current value
Document	Vanni Ride Master Specification
Version	2.0
Status	Current project authority
Updated	14 October 2026
Repository	IT3162-Group-6 / Project-VanniRide
Baseline	Commit 6c1719b on remote main and frontend-backend-integration
Live frontend	https://vanni-ride.pages.dev
Live backend	https://project-vanniride-production.up.railway.app
Database	MongoDB Atlas database vanniRideDB
Audience	Project members, assessors, reviewers, and future maintainers

Authority and Precedence

1. This Version 2.0 master specification is the primary requirements and behavior authority.
2. The code at the baseline commit is the authority for exact implemented endpoint behavior and validation details.
3. The canonical integration contract, API guide, database documents, and deployment guides remain supporting references.
4. If a supporting document conflicts with this specification or the current code, the team must resolve the conflict and update the documentation before merging further work.
5. The old pre-development specification remains historical evidence only and is no longer the active source of unresolved decisions.

Change Control

- Major scope, status, pricing, permission, database, API, cancellation, or deployment changes require team agreement.
- Every approved change must be documented, tested at the appropriate level, and reviewed before release.
- Secrets and administrator passwords must remain in secure service configuration, never in Git or project reports.
- Production-bound changes currently need to reach both main and frontend-backend-integration because Railway is connected to the integration branch.

Contents

- 1 Project Overview and Current State
- 2 Product Scope
- 3 User Roles and Permissions
- 4 System Architecture and Technology
- 5 Authentication Security and Accounts
- 6 Customer Experience
- 7 Rider Experience
- 8 Administrator Experience
- 9 Ride and Delivery Request Specification
- 10 Maps Address Lookup Routing and Fare
- 11 Ride Lifecycle and Availability
- 12 Cancellation Rules
- 13 Payment Ratings Chat and Notifications
- 14 Data Model
- 15 HTTP API and Real Time Contract
- 16 Deployment and Operations
- 17 Testing Quality and Definition of Done
- 18 GitHub and Team Workflow
- 19 Remaining Work and Deferred Features
- Appendix A Quick Pick Coordinates
- Appendix B Canonical Values
- Appendix C Major Changes from the Pre Development Specification

1 Project Overview and Current State

Vanni Ride is a student-powered transport and delivery platform for the University of Vavuniya community. It connects customers who need a ride or a local delivery with approved student riders who own motorcycles. Administrators supervise accounts, rider approval, safety operations, payments, chat access, and platform activity.

The project has moved beyond the original pre-development stage. A working full-stack application is deployed with a React frontend, Node.js backend, MongoDB database, role-based dashboards, route and fare calculation, structured delivery ordering, real-time updates, persistent notifications, and administrative controls.

1.1 Current Baseline

Area	Current state
Application	Working version one with subsequent fixes and feature refinements
Source baseline	Commit 6c1719b
Frontend	Deployed through Cloudflare Pages
Backend	Deployed through Railway and connected to MongoDB Atlas
Maps	OpenStreetMap and Leaflet with backend geocoding and road routing
Live communication	Socket.IO events with timed refresh recovery
Verification	Focused routing and geocoding tests pass; live backend health reports database connected
Pending	Complete team end-to-end review and public-release hardening

1.2 Main Objectives

- Allow a customer to request either transportation or delivery using map-based pickup and destination locations.
- Calculate a road-route distance, duration, and estimated cash charge before confirmation.
- Allow an approved available rider to review, ignore, restore, and atomically accept available requests.
- Provide controlled ride progress, cancellation, chat, payment, rating, history, and notification workflows.
- Give administrators auditable tools for user management, rider approval, safety intervention, payment correction, and platform communication.
- Operate as an academic demonstration using free services and without requiring paid third-party APIs.

2 Product Scope

2.1 Included in the Current Version

Area	Implemented scope
Accounts	Registration, login, logout, profiles, JWT sessions, university-email restriction, role-based access
Transport	Map-based ride request, road-route fare preview, rider acceptance, progress, history
Delivery	Food, water, and other parcel ordering with category-specific data and pricing
Rider operations	Vehicle profile, admin approval, availability, ignore/restore requests, Google Maps navigation
Cancellation	Immediate pre-start cancellation, mutual started cancellation, timeout, allowance, admin force cancellation
Payment	Cash-only pending and paid records, rider confirmation, admin correction
Communication	Ride-scoped text chat, conversation list and filtering, temporary post-completion access
Notifications	Persistent notification center, unread badge, real-time delivery, admin broadcasts
Ratings	One customer rating per completed request and rider rating summaries
Administration	Users, riders, rides, cancellations, payments, ratings, statistics, audits
Deployment	Cloudflare Pages, Railway, MongoDB Atlas, free mapping and routing services

2.2 Explicitly Excluded or Deferred

- Online card, bank, wallet, or payment-gateway processing.
- Continuous GPS tracking or an in-app turn-by-turn navigation engine.
- Image, file, voice, or video messages.
- Automatic purchase-price calculation for food or goods; delivery pricing covers rider service only.
- Public registration of administrator accounts.
- Unrestricted operation outside the intended Northern Province service area.
- Paid infrastructure, paid map APIs, paid custom domains, or paid monitoring services at this stage.
- A completed production-grade end-to-end certification; team review remains pending.

2.3 Product Constraints

- Cash is the only payment method.
- A customer may have one active transport request and one active delivery request at the same time.
- Only approved riders can become available and accept work.
- The system depends on free external map, geocoding, routing, hosting, and database quotas.
- The public application must not expose the routing API key, database connection string, JWT secret, or administrator credentials.

3 User Roles and Permissions

Capability	Customer	Rider	Administrator
Register publicly	Yes	Yes	No
Create ride or delivery	Own requests	No	No
Accept available request	No	Approved and available	No
Update progress	No	Assigned request only	Safety cancellation only
Chat	Assigned rider	Assigned customer	Audited read-only view
Confirm cash	View status	Assigned completed request	Audited correction
Rate rider	Once after completion	View own summary	View all ratings
Manage accounts	Own profile	Own profile and vehicle	Suspend or reactivate
Notifications	Receive	Receive	Receive and broadcast

3.1 Customer Rules

- Customers can create, view, track, cancel, and review their own requests only.
- Customers see assigned rider information after acceptance and can communicate while chat access is valid.
- Customers can view cancellation allowance, request history, cash status, and ratings submitted.
- Customers cannot accept work, change rider lifecycle status, manage users, or access administrator endpoints.

3.2 Rider Rules

- New riders submit vehicle type, model, registration number, and colour and begin as pending and unavailable.
- Approved riders can set themselves available or unavailable when they are not busy.
- A rider can accept only a still-requested ride while approved and available; acceptance is atomic.
- Ignoring is reversible and personal: the request moves to the bottom and is greyed, but remains acceptable later.
- Riders can update only their assigned request and cannot skip lifecycle states.

3.3 Administrator Rules

- Administrators use pre-created protected accounts; their credentials are managed outside source control.
- Every sensitive mutation requires an administrative reason and produces an append-only audit entry.
- Administrators can approve or reject riders, manage account status, review chat access, force-cancel active requests, and correct completed-request payment records.
- Administrators cannot rewrite completed or cancelled ride history and cannot use the admin panel as a customer or rider.

4 System Architecture and Technology

The system uses a conventional layered web architecture with browser clients calling an authenticated API and receiving real-time events from the same backend.

Users -> React frontend -> Express API and Socket.IO -> MongoDB Atlas

Layer	Technology	Responsibilities
Frontend	React, Vite, React Router	Role dashboards, forms, validation, state, lazy-loaded pages
Maps	Leaflet and OpenStreetMap	Map display, markers, route geometry, attribution
Backend	Node.js and Express	API, authorization, validation, workflows, external service adapters
Real time	Socket.IO	Ride, chat, cancellation, approval, and notification events
Database	MongoDB with Mongoose	Persistent records, constraints, indexes, audit history
Frontend hosting	Cloudflare Pages	Static Vite build and single-page application hosting
Backend hosting	Railway	Node service, WebSocket support, health checks, environment secrets
Database hosting	MongoDB Atlas	Hosted vanniRideDB database
Routing	HeiGIT OpenRouteService with public OSRM fallback	Road route distance, duration, and geometry
Geocoding	HeiGIT Pelias with Nominatim fallback	Search suggestions and coordinate-to-address lookup
External navigation	Google Maps URL	Rider turn-by-turn navigation without a Google API key

4.1 Repository Structure

- **frontend/** React pages, components, data, contexts, API client, tests, and production build configuration.
- **backend/** Express routes, controllers, models, middleware, services, workers, scripts, and automated tests.
- **database/** Collection creation, migrations, validators, indexes, sample data, and common queries.
- **documentation/** Integration, API, database, deployment, testing, and this master specification.

5 Authentication Security and Accounts

5.1 Registration and Login

- Public registration supports CUSTOMER and RIDER only.
- Production registration requires the configured University of Vavuniya email domain, currently vau.ac.lk.
- A non-university-email bypass may be enabled for local testing only and is rejected in production configuration.
- Passwords are stored as bcrypt hashes. Login returns a JWT and the frontend sends it as a Bearer token.
- Logout increments the token version so the current session becomes invalid server-side.

- Suspended accounts and revoked, expired, or malformed tokens are rejected by both HTTP and Socket.IO authorization.

5.2 Security Controls

Control	Implemented behavior
Role authorization	Backend middleware checks stored role; hiding frontend controls is not treated as security
CORS	Only explicitly configured browser origins are accepted in production
Request limits	JSON and form bodies are limited to 100 KB
Headers	No-sniff, frame denial, and referrer protections are applied
Secrets	Database URI, JWT secret, and routing key are environment variables
Runtime validation	Production startup rejects unsafe or missing critical configuration
Administrative accountability	Sensitive operations store actor, reason, before state, after state, and timestamp

5.3 Test and Administrator Accounts

Local demonstration accounts may be seeded only in disposable local databases. Production administrator accounts are created with protected setup tooling and strong credentials stored privately. The project currently has three pre-created administrator accounts for controlled testing; their passwords must not appear in Git, screenshots, reports, or this specification.

6 Customer Experience

- Dashboard summary for active transport, active delivery, cash history, and cancellation allowance.
- Separate sidebar actions for Request a Ride and Request a Delivery.
- Map-first location selection with pickup and destination summary boxes, search, quick picks, draggable markers, and browser current location.
- Three-step request flow: location and details, route and fare summary, confirmation.
- Active Request view with route map, rider information, request status, cancellation controls, and live updates.
- Messages workspace with conversation list, filtering, and ride-scoped chat.
- Request history with cash status, cancellation information, rating action, and completed-chat-access request.
- Notification center with all and unread filters, mark-one and mark-all read behavior, and deep links.
- Profile editing and secure logout.

6.1 Active Request Progress

After a rider accepts, customer progress is updated through Socket.IO ride events. A timed refresh remains as recovery when the live connection is interrupted. The rider advances the request from ACCEPTED to ARRIVED, STARTED, and COMPLETED; the customer cannot manually change these states.

7 Rider Experience

- Dashboard showing approval, availability, active work, earnings, pending receipts, and rating summary.
- Vehicle profile management; changing approved vehicle details returns the rider to pending review and unavailable status.
- Available Requests list visible only when approved and available.
- Ignore action moves a request to the bottom and greys it; restore makes it normal again; ignored requests remain acceptable.
- Atomic acceptance prevents two riders from taking the same request.
- Request details show map, pickup, destination, distance, estimated charge, customer and delivery information.
- Active Request view supplies Google Maps navigation to pickup before arrival and destination afterward.
- Status progression, cancellation response, chat, history, and cash-confirmation actions are restricted to the assigned request.

7.1 External Navigation

Vanni Ride does not implement turn-by-turn navigation. It opens Google Maps using a standard directions URL built from stored coordinates. This current integration does not use a Google Maps API key and does not retrieve Google tracking data back into Vanni Ride. Request status remains controlled inside Vanni Ride.

8 Administrator Experience

Area	Administrator capability
Dashboard	View platform counts and operational summaries
Users	List users; suspend or reactivate accounts with a reason
Riders	Review vehicle profiles; approve or reject applications
Rides	List and inspect requests; audit conversation access; force-cancel active work
Payments	Review cash records and correct documented completed-request disputes
Chat access	Approve or reject customer requests for 24-hour post-completion chat
Ratings	Review platform ratings
Notifications	Broadcast app updates to customers, riders, or everyone
Audit	Record every sensitive action and reason

Administrator message viewing is read-only and requires a reason. Conversation content is not copied into the audit log; the log records operational metadata. Force cancellation cannot modify completed or already-cancelled requests and does not consume participant cancellation allowance.

9 Ride and Delivery Request Specification

9.1 Common Request Data

- Customer identity from the authenticated session.
- Request type TRANSPORT or DELIVERY.
- Pickup and destination objects containing address, latitude, and longitude.
- Road distance, estimated duration for preview, route geometry for display, and stored estimated cash charge.
- Lifecycle status and timestamps.
- Assigned rider after acceptance.
- One pending cash payment created with the request.

9.2 Delivery Categories

Category	Required details	Receiver fields
FOOD	Outlet name and one or more food item lines with type and quantity; optional item description	Not required
WATER	One or more water container lines with size and quantity	Not required
PARCEL	Parcel type, weight, optional notes, and custom type when Other is chosen	Receiver name and Sri Lankan mobile number required

9.3 Food Order Values

Vegetable meal, chicken meal, salmon meal, fish meal, egg meal, sausages meal, meatball meal, noodles, kottu, pasta, parata, thosai, and Other food.

- Food lines are dynamic: customers may add and remove items.
- Quantity uses plus and minus controls and must be a whole number from 1 through 20 per line.
- Each line may contain an optional description up to 300 characters.
- Other food requires a custom item name.
- A request may contain up to 20 item lines.

9.4 Water Order Values

20L bottle, 10L bottle, 5L bottle, less than 5L bottle, and Other water container.

- Water lines are dynamic and use quantity controls from 1 through 20.
- Other water container requires a description.
- A request may contain up to 20 item lines.

9.5 Other Parcel Values

Documents, food, clothing, medicine, electronics, and Other parcel.

- Weight is recorded from 0.1 kg through 50 kg in 0.1 kg precision.
- Other parcel requires a custom type.
- Additional notes are optional up to 500 characters.

- Receiver name and valid Sri Lankan mobile number are mandatory only for this category.
- No parcel quick-pick locations are defined in the current version.

10 Maps Address Lookup Routing and Fare

10.1 Location Selection

- The map uses OpenStreetMap tiles through Leaflet and retains visible attribution.
- Clicking the pickup or destination summary box changes which location the next map click will set.
- The Pickup and Destination tabs retain the same selection function.
- Markers can be dragged to correct a selected point.
- Browser location can be used as pickup when permission is granted.
- Search is limited to Sri Lanka and bounded to the intended Northern Province area.

10.2 Address Lookup

The backend first uses the configured HeiGIT Pelias geocoder. If that service fails, the application falls back to public Nominatim for both search and reverse lookup. Coordinates remain the authoritative location values; the readable address is presentation data. Search results are cached and provider requests are throttled.

10.3 Road Routing

The backend first requests a driving route from the HeiGIT OpenRouteService directions endpoint. It selects the shortest valid route returned by the provider and returns distance, estimated duration, and GeoJSON geometry. If the primary service fails, the backend uses the configured public OSRM-compatible fallback. A request is not created when no valid drivable route is available from either provider.

Route preview writes no database data. Request creation recalculates the route and price on the server so a client cannot forge the charge.

10.4 Fare and Delivery Charge

Request	Calculation
Transport	LKR 200 base + LKR 80 x road kilometres
Food delivery	LKR 150 base + LKR 60 x road kilometres + LKR 25 per food parcel
Water delivery	LKR 150 base + LKR 60 x road kilometres + LKR 30 per container
Other parcel	LKR 150 base + LKR 60 x road kilometres + LKR 15 per started kilogram

The total is rounded to the nearest whole LKR and stored on both the ride and its pending payment. Food, water, medicine, or other product purchase prices are not included; the estimate is the rider's service charge only.

11 Ride Lifecycle and Availability

REQUESTED -> ACCEPTED -> ARRIVED -> STARTED -> COMPLETED

Status	Meaning	Primary actor
REQUESTED	Customer confirmed the request; available riders may review it	Customer
ACCEPTED	One approved available rider accepted atomically	Rider
ARRIVED	Rider reached the pickup point	Assigned rider
STARTED	Transport or delivery is in progress	Assigned rider
COMPLETED	Service finished; cash may be confirmed and rating becomes available	Assigned rider
CANCELLED	Service ended through an allowed cancellation path	Participant or admin

11.1 Rider Availability

Value	Rule
UNAVAILABLE	Rider is offline, pending/rejected, or voluntarily not accepting requests
AVAILABLE	Approved active rider may view and accept requests
BUSY	Rider has an accepted active request and cannot accept another

Acceptance changes AVAILABLE to BUSY. Completion or final cancellation releases the rider back to AVAILABLE according to the workflow.

12 Cancellation Rules

12.1 Before Start

- A customer may immediately cancel their REQUESTED or ACCEPTED request with a reason.
- The assigned rider may immediately cancel an ACCEPTED request with a reason.
- Participant cancellation is not offered in ARRIVED state.
- The request is retained as CANCELLED; it is never deleted.

12.2 After Start

1. Either participant submits a reasoned cancellation request while the ride remains STARTED.
2. The other participant is notified and chooses Cancel or Resume.
3. Cancel finalizes the request as CANCELLED using mutual cancellation.
4. Resume keeps the ride STARTED and resolves the request without consuming a final cancellation.
5. If no response arrives within 15 minutes, the persisted worker automatically cancels the request.

12.3 Rolling Allowance

- A user may initiate at most five final cancellations in a rolling 60-minute window.
- Transport and delivery share the same allowance.
- A pending started cancellation reserves one place until it is resolved.
- Resumed requests release the reservation and do not consume the allowance.
- Administrator force cancellation never consumes participant allowance.
- The API exposes limit, used, remaining, pending reservations, and the next reset time.

13 Payment Ratings Chat and Notifications

13.1 Cash Payment

- Every request creates one payment record with method CASH and status PENDING.
- Only the assigned rider may mark cash PAID after the request is COMPLETED.
- Confirmation records the rider and paid timestamp and notifies the customer.
- Rider earnings count PAID completed-request payments; pending completed receipts are shown separately.
- An administrator may make an audited correction for a documented completed-request dispute.

13.2 Ratings

- Only the customer may rate the assigned rider after completion.
- Each completed request may be rated once from 1 through 5.
- A review is optional and may contain up to 1,000 characters.
- Rider views show the average and total rating count; the average is absent when no rating exists.

13.3 Chat

- Only the request customer and assigned rider may read and send ride messages.
- Sending is allowed during ACCEPTED, ARRIVED, and STARTED.
- Completed and cancelled conversations remain readable; cancelled conversations remain permanently read-only.
- A customer may request temporary chat reopening for a completed request by giving a reason.
- Admin approval reopens sending to both participants for 24 hours; history is never deleted when access expires.
- Messages are text only, non-empty, and limited to 1,000 characters.
- The frontend presents a conversation list with filters and opens the selected conversation.

13.4 Notification System

Notifications are persistent MongoDB records and are also delivered live through Socket.IO. The top-bar bell displays an unread count, refreshed after live events, window focus, and a 20-second recovery interval. Users can filter All or Unread, open linked actions, mark one read, or mark all read.

Notification type	Typical trigger
RIDE_REQUEST	New request available to approved and available riders

Notification type	Typical trigger
RIDE_ACCEPTED	Customer request accepted
RIDE_STATUS	Arrival, start, or completion
RIDE_CANCELLED	Participant or administrator cancellation
CANCELLATION_REQUESTED	Started-request confirmation required
CANCELLATION_RESOLVED	Cancel, resume, or automatic timeout result
CHAT_MESSAGE	New message for the other participant
CHAT_ACCESS	Access review requested, approved, or rejected
RIDER_APPROVAL	Rider approval decision
PAYMENT_CONFIRMED	Rider confirmed cash
SYSTEM	Administrator app update or announcement

14 Data Model

MongoDB automatically supplies ObjectId values. API fields use camelCase; database fields use snake_case. The current schema contains twelve application collections.

Collection	Purpose and principal fields
users	Identity, email, phone, password hash, role, account status, token version, timestamps
riders	User reference, availability, vehicle, approval decision, review metadata
rides	Participants, type, delivery details, locations, road distance, fare, status, lifecycle timestamps
payments	Unique ride reference, amount, CASH method, PENDING or PAID, confirmation metadata
messages	Ride, sender, text, sent time
cancellations	Final cancellation actor, reason, previous status, mode, timestamp
cancellation_requests	Started-request mutual cancellation, responder, deadline, resolution
ratings	Unique ride rating, customer, rider, score, review
chat_access_requests	Completed-chat request, admin review, 24-hour access window
admin_audit_logs	Admin action, target, reason, before and after values, timestamp
ignored_ride_requests	Unique rider and request pair plus ignored timestamp
notifications	User, type, title, message, internal link, ride, read state, timestamp

14.1 Core Relationships

- One user may have one rider profile when role is RIDER.
- One customer creates many rides; a ride has zero or one assigned rider.
- One ride has one payment, zero or one final cancellation, zero or one rating, and many messages.
- One ride may have cancellation requests and chat access requests over time, subject to active uniqueness rules.
- One user may receive many notifications and produce many audited or participant actions.

14.2 Required Integrity Controls

- Unique normalized user email and rider vehicle registration number.
- Unique rider profile per user, payment per ride, final cancellation per ride, rating per ride, and ignored rider-request pair.
- Indexes for active request discovery, rider approval, messages, cancellation windows, chat expiry, notifications, and audits.
- MongoDB validators enforce role, status, delivery structure, phone, rating, and lifecycle-compatible values.

15 HTTP API and Real Time Contract

Private endpoints require Authorization Bearer token. Successful responses use success true and a data object; failures use success false, a machine-readable code when available, and a human-readable message.

Category	Current endpoints
Authentication	POST /api/auth/register, /login, /logout
Users	GET or PUT /api/users/profile; GET /history and /cancellation-allowance
Rider profile	PATCH /api/users/rider/availability; PUT /rider/profile; GET /rider/earnings
Maps	GET /api/maps/search and /reverse; POST /route-preview
Notifications	GET /api/notifications; PATCH /read-all and /:notificationId/read
Rides	POST or GET /api/rides; available, details, ignore, restore, accept, status, cancel, cancellation request
Chat	GET or POST /api/rides/:rideId/messages; POST chat-access-requests
Payment	GET or PATCH /api/rides/:rideId/payment
Ratings	POST /api/rides/:rideId/rating; GET /api/riders/:riderId/ratings
Admin	Users, riders, rides, messages, cancellation, payments, ratings, chat access, statistics, broadcasts

15.1 Expected Status Codes

200 read or update; 201 creation; 202 cancellation awaiting confirmation; 400 invalid data; 401 invalid session; 403 unauthorized or suspended; 404 not found; 409 lifecycle, duplicate, availability, or quota conflict; 429 provider rate limit; 503 map or routing provider unavailable.

15.2 Socket.IO Events

Direction	Events
Client to server	join_ride, send_message
Server to client	new_ride_request, ride_status_changed, new_message, cancellation_requested, cancellation_resolved, chat_access_updated, rider_approval_updated, ride_force_cancelled, notification_created

Socket room membership never replaces database authorization checks. Session validity is checked when connecting, joining a ride, and sending messages.

16 Deployment and Operations

Service	Current deployment	Operational note
Frontend	Cloudflare Pages	https://vanni-ride.pages.dev
Backend	Railway	https://project-vanniride-production.up.railway.app
Database	MongoDB Atlas	vanniRideDB; credentials remain private
Routing and geocoding	HeiGIT primary	Free API key stored only in backend environment
Fallback routing	Public OSRM-compatible service	Used when primary routing fails
Fallback geocoding	Public Nominatim	Used when primary address lookup fails

16.1 Deployment Branches

- Remote main and frontend-backend-integration currently point to the same baseline commit.
- Railway deploys the backend from frontend-backend-integration.
- The frontend is connected to the GitHub repository through Cloudflare Pages.
- Until hosting branch configuration is simplified, production-ready changes must be kept aligned on both branches.

16.2 Required Backend Configuration

NODE_ENV, PORT, MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN, UNIVERSITY_EMAIL_DOMAIN, ALLOW_NON_UNIVERSITY_EMAILS, CORS_ORIGINS, map-provider URLs and key, routing profile and timeout, geocoding throttle and cache values, and cancellation sweep interval.

The frontend production build requires VITE_API_URL pointing to the Railway backend plus /api, with mock authentication and mock data disabled.

16.3 Operational Checks

- GET /health must return HTTP 200 with database connected.
- Cloudflare must serve nested React routes after refresh.

- Configured CORS must permit only the live frontend and authorized local development origins.
- Socket.IO must reconnect after host sleep or temporary network loss.
- Logs must contain no passwords, tokens, API keys, or connection strings.
- Database migrations, validators, and indexes must be applied before relying on new fields.

17 Testing Quality and Definition of Done

17.1 Required Test Levels

Level	Purpose
Unit	Fare, delivery validation, routing, geocoding, state, and utility behavior
API	Validation, roles, permissions, lifecycle conflicts, and response contracts
Integration	Frontend, backend, Socket.IO, external adapters, and MongoDB working together
Functional	Complete customer, rider, and administrator journeys
Deployment smoke	Health, login, map, route, notifications, nested routes, and database connectivity
Security and negative	Unauthorized roles, suspended users, invalid tokens, conflicting acceptance, invalid status changes

17.2 Current Verification Position

- The integration audit previously passed backend, frontend, lint, build, dependency, database, and local role-login checks at that baseline.
- Subsequent focused tests verify the current primary and fallback routing and geocoding behavior.
- The live Railway health endpoint currently reports the server ready and MongoDB connected.
- A complete post-change end-to-end team review has intentionally not yet been performed and remains required.

17.3 Definition of Done

1. The implementation follows this master specification or an approved documented change.
2. Relevant automated tests pass, including authorization and negative cases.
3. Frontend lint and production build pass for frontend changes.
4. MongoDB validators and indexes accept the resulting data for schema changes.
5. The feature is integrated and exercised in its user journey.
6. Errors and empty states are understandable to users and do not expose technical secrets.
7. Documentation and API examples are updated.
8. Another member reviews the change before production merge or deployment.
9. No unresolved critical defect remains.

18 GitHub and Team Workflow

- Create a feature branch for a defined change; do not place unfinished work directly on main.
- Keep commits focused and describe the actual code change, not a member assignment or assessment strategy.
- Pull or fetch before merging and resolve conflicts against the current shared contract.
- Run relevant tests and review the diff before pushing.
- Use a pull request and another member's review for substantial work.
- After approval, merge to main and ensure Railway's deployment branch receives the same production backend commit.
- Cloudflare Pages and Railway redeploy automatically from their configured GitHub branches.
- Never commit .env files, connection strings, passwords, JWT secrets, API keys, or production test data.

18.1 Team Responsibilities

The six-member team may divide work across frontend, backend, database, testing, integration, and documentation, but ownership does not remove shared review responsibility. Database field changes, API format changes, lifecycle changes, and major feature additions must be communicated across the whole group.

19 Remaining Work and Deferred Features

19.1 Required Before an Actual Public Release

- Perform the postponed complete end-to-end team review across customer, rider, and administrator journeys.
- Confirm all newly created and predefined accounts can authenticate in the deployed environment and rotate any temporary credentials.
- Recheck every ride and delivery category, route preview, request creation, acceptance, lifecycle, cancellation, cash confirmation, rating, chat, and notification path.
- Confirm free-tier quotas, acceptable-use policies, cold-start behavior, and fallback-provider limits for expected assessment usage.
- Establish an Atlas backup or export process and review database access restrictions.
- Add suitable rate limiting for login and public-facing endpoints without requiring paid infrastructure.
- Decide on free log retention and alerting and document incident handling.
- Standardize the production branch so one approved merge updates each intended deployment without manual branch duplication.

19.2 Optional Improvements

- Continuous integration for tests, lint, build, audits, and migration checks on pull requests.
- Further frontend bundle splitting if initial load performance becomes a problem.
- Additional accessibility testing and keyboard-navigation review.
- More comprehensive provider telemetry and user-friendly degraded-service messages.
- A formal admin credential rotation procedure and recovery runbook.

19.3 Features Still Out of Scope

- Paid services unless the team later approves funding.
- In-app continuous navigation or GPS tracking.
- Automatic retrieval of live Google Maps tracking data.
- Online financial transactions.
- Media and file sharing in chat.

Appendix A Quick Pick Coordinates

A.1 Ride Locations

Location	Latitude	Longitude
University of Vavuniya	8.758650	80.410810
Vavuniya Town	8.751400	80.497100
Vavuniya Bus Stand	8.753550	80.497590
Vavuniya Railway Station	8.754280	80.493780
Poovarasankulam Hospital	8.752600	80.367350
Hostel Premises Pambaimadu	8.754480	80.399810

A.2 Food Pickup Locations

Location	Latitude	Longitude
Hostel Canteen	8.754679431	80.399979593
Rilvan Restaurant	8.755848949	80.391798544
RS Restaurant	8.758562832	80.406021693
Army Camp Canteen	8.759727071	80.421441889
Applied Canteen	8.755239887	80.410100485
Business Studies Canteen	8.756951362	80.410784838

A.3 Water Pickup Locations

Location	Latitude	Longitude
Girls Hostel Water Outlet	8.755949051	80.399492566
BS Canteen Water Outlet	8.756834249	80.410776474

These coordinates are hardcoded convenience choices. Users may still choose any valid pickup and destination within the supported map workflow.

Appendix B Canonical Values

Domain	Allowed values
User role	CUSTOMER, RIDER, ADMIN
Account status	ACTIVE, SUSPENDED
Request type	TRANSPORT, DELIVERY
Delivery category	FOOD, WATER, PARCEL
Ride status	REQUESTED, ACCEPTED, ARRIVED, STARTED, COMPLETED, CANCELLED
Rider availability	AVAILABLE, UNAVAILABLE, BUSY
Rider approval	PENDING, APPROVED, REJECTED
Payment method	CASH
Payment status	PENDING, PAID
Cancellation mode	IMMEDIATE, MUTUAL, AUTO_TIMEOUT, ADMIN_FORCE
Cancellation request	PENDING, CONFIRMED, RESUMED, AUTO_CANCELLED, ADMIN_CANCELLED
Chat access request	PENDING, APPROVED, REJECTED, EXPIRED
Notification	RIDE_REQUEST, RIDE_ACCEPTED, RIDE_STATUS, RIDE_CANCELLED, CANCELLATION_REQUESTED, CANCELLATION_RESOLVED, CHAT_MESSAGE, CHAT_ACCESS, RIDER_APPROVAL, PAYMENT_CONFIRMED, SYSTEM

Appendix C Major Changes from the Pre Development Specification

Earlier provisional area	Final implemented decision
Transport or delivery not finalized	Both are implemented; delivery has Food, Water, and Other Parcel tabs
Delivery item details uncertain	Structured dynamic food and water lines plus receiver and parcel data
Complex fare estimation optional	Server-side road-route pricing with delivery handling charges
Rider reject or ignore unclear	Reversible ignore only; item moves down and remains acceptable
Map scope unspecified	Leaflet, OpenStreetMap, bounded search, quick picks, markers, route geometry
Navigation unspecified	Free redirect to Google Maps; no in-app continuous navigation
Cancellation after start unresolved	Mutual confirmation, 15-minute automatic cancellation, five-per-hour allowance
Cash confirmation unresolved	Assigned rider confirms after completion; admin may audit-correct
Chat after completion unresolved	Read-only history plus customer-requested 24-hour admin approval
Rider eligibility basic	Vehicle profile and administrator approval are mandatory

Earlier provisional area	Final implemented decision
Notification system absent	Persistent, real-time notification center and admin broadcasts
Admin access only conceptual	Protected pre-created accounts and audited operational controls
Deployment not selected	Cloudflare Pages, Railway, MongoDB Atlas, and free map providers
Provider failures blocked core flow	Routing and address lookup now have approved free fallbacks

Supporting Repository References

- documentation/integration/canonical-contract.md
- documentation/integration/final-integration-audit.md
- documentation/integration/deployment-readiness.md
- documentation/integration/free-deployment-runbook.md
- documentation/api/backend-b2-api.md
- documentation/database/database-design.md
- database/README.md
- frontend/README.md
- backend/README.md

Approval Record

The team should record formal approval in its normal project review process. Approved future changes must update the version, date, baseline commit, and affected sections of this master specification.

Role	Name	Decision	Date
Project representative			
Frontend reviewer			
Backend reviewer			
Database and testing reviewer			